

Rammevilkår

Følgende rammebetingelser gjelder for eksamen, og det er viktig at du setter deg grundig inn i dem før du starter arbeidet. Sørg for at alle krav og retningslinjer følges nøye gjennom hele prosessen.

Kilder

All kode du ikke selv har skrevet skal tydelig oppgis med kilder. Dette gjelder også AI-generert kode. Du må også gjøre rede for hvorfor du tok det inn i kodebasen, skriv dette i samme Readme-fil som nevnes senere.

Kvalitet på kode

Det legges stor vekt på strukturert og ryddig programmering. Koden din skal være oversiktlig og lett å forstå, samtidig som den tilfredsstiller alle funksjonelle krav som er spesifisert i oppgaven.

Design og brukeropplevelse

Applikasjonen skal ha et visuelt design som følger anerkjente retningslinjer for mobilapper. Du forventes å bruke Assets-funksjonen i Xcode til å organisere og optimalisere grafiske ressurser som farger, ikoner og bilder. Godt design handler ikke bare om estetikk, men også om å skape en intuitiv og brukervennlig opplevelse for brukeren.

Feilhåndtering

Det forventes at alle steder som appen kan feile (eks: nettverksfall), så skal man vise en forståelig feilmelding til brukeren.

Krav til leveranse

Når prosjektet er fullført, skal det kunne kompileres og kjøres direkte i iPhone-simulatoren uten endringer. Komprimer hele prosjektmappen til en ZIP-fil før du laster den opp i WISEflow. Kontroller at alle nødvendige filer er inkludert før innlevering. dobbeltsjekk dette! Husk også å legge ved videofilen.

Presentasjon av prosjektet

I tillegg til selve appen skal du lage en kort presentasjonsvideo der du demonstrerer applikasjonen i simulatoren. Videoen skal også inneholde en forklaring av sentrale deler av koden din. Dette gir sensor innsikt i dine valg og vurderinger, og viser at du har forstått både tekniske og designmessige aspekter ved prosjektet. Her skal du forklare mens du ser på koden, ikke lese fra et ferdigskrevet manus.

Navigasjon

De ulike skjermene som nevnes under skal vises i en tab-meny.

Uklarheter

Noe uklart i oppgaven? Gjør dine egne vurderinger og forutsetninger og skriv disse i en readmefil du legger ved prosjektet.

Oppgave 1 Søk etter forfattere

Lag en SwiftUI-app som lar brukeren søke etter forfattere via Open Library-search-API-et:

- Bruk endepunktet <https://openlibrary.org/search/authors.json?q=<søkeord>>.
- Vis relevante felter i søkeresultatet: `name`, `birth_date` (hvis finnes), `top_work`, `work_count` etc.
- Implementer søkefelt med debounce, slik at API-et ikke kalles for hvert tastetrykk.
- Lag en gjenbrukbar nettverkslayer (`URLSession/async-await`) med ordentlig feilhåndtering og indikatorer for lasting/feil. Denne skal brukes på alle nettverkskall.

Oppgave 2 Forfatterdetaljer

Når brukeren velger en forfatter fra søkelisten:

- Når brukeren velger en forfatter, henter appen detaljer via <https://openlibrary.org/authors/<KEY>.json>
- Presenter informasjonen pent i en detaljvisning: biografi, fødsels-/dødsår, topp-sjanger m.m. (håndter at noen felt kan mangle).
- Hent alle forfatterbilder og vis de på detaljsiden i en skrollbar horizontal liste. Med en gang man kommer inn på detaljsiden skal det vises en firkant med samme størrelse som bildene du skal vise, for hvert kommende bilde man venter på at laster ned, men en spinner i. Vis den helt til bildet er ferdig lastet ned og vises.
- Som siste element i den skrollbare horisontale lista, skal man vise en pluss-knapp som lar bruker ta bilde eller velge fra bibliotek og legge til flere bilder, disse skal persisteres så de lagres mellom hver gang appen restarteres.
- Du kan anta at brukeren gir tilgang til bilder - trenger ikke håndtere caset hvor brukeren sier nei til å gi tilgang til kamera/library. Husk også å legg inn i Plist keys så du har lov å spørre, `NSCameraUsageDescription` og `NSPhotoLibraryUsageDescription`.

Oppgave 3 Liste over verk

- Bruk <https://openlibrary.org/authors/<KEY>/works.json> for å hente en liste over verk skrevet av forfatteren.
- Default gir API-et 50 verk, men du skal legge til paginering med `?offset=`.
- Hver gang man kommer til bunnen av lista skal det automatisk laste de neste 50 verkene. Disse skal vises som de neste 50
- Ha en knapp som lar brukeren sortere på ulike kriterier: tittel, publikasjonsår, popularitet

Oppgave 4 Splash-screen

- Etter appen starter skal man vise en animert splash-skjerm som er fullskjerm. Den skal vises uavhengig hvor lang APIkallet på første side varer.
- Når 5 sekunder har gått skal man vise den første Tab.

- Animasjonen skal være nøyaktig som beskrevet:
 - En bok (bruk emoji) skal være midtstilt med en sirkel rundt seg. Boka skal pulserer mellom 0,8 og 1,2 størrelse.
 - Boka skal være sånn at den dekker ca $\frac{1}{3}$ av skjermen i bredden
 - Det skal regne ulike bokstaver fra toppen av skjermen og fade gradvis ut mens de når bunnen. Bruk alfabetet og velg tilfeldige ulike bokstaver, i hvert fall 10 på skjermen samtidig.
 - Under boka skal det stå navnet på appen, med regnbuefarget font. Lag din egen custom view modifier.

Oppgave 5 - Video

- Ta opp en video hvor du går gjennom kodebasen din og forklarer med egne ord, her er det ikke meningen at man bare skal lese opp fra et ferdigskrevet manus. Maks 7 min. Dette skal dekkes i videoen:
 - Nettverkskall i de ulike oppgavene
 - Håndteringen av forfatterbilder og mulighet for å laste opp egne bilder
 - Paginering og listevissning av verk
 - Animasjoner i SwiftUI og Splash screen